Why Naive RAG Fails: An Empirical Study of Knowledge Injection Strategies for Large Language Model Reasoning

Sathvik A R
PES University
arsathvik48@gmail.com

***Abstract*—Large language models (LLMs) exhibit knowledge gaps on counterintuitive facts, motivating the use of retrieval-augmented generation (RAG) and related techniques. We conduct a systematic empirical evaluation of five knowledge injection strategies—zero-shot prompting, chain-of-thought (CoT), self-consistency (SC), naive RAG, and knowledge-base augmented SC with structured Step 1/Step 2 prompting (KB+SC+Step1/2)—on a 50-question benchmark using MiMo-v2.5-Pro. Our central finding is that naive RAG degrades accuracy by 14 percentage points (56.0% vs 70.0% zero-shot), demonstrating that unstructured knowledge injection is counterproductive. Chain-of-thought prompting similarly hurts performance (60.0% vs 70.0%), while self-consistency alone causes a severe 24-point regression (46.0% vs 70.0%). Only our structured KB+SC+Step1/2 method achieves significant improvement over zero-shot (82.0%, +12pp, $p = 0.0019$). We identify the root cause: the bottleneck for LLMs on knowledge-sensitive question answering is not reasoning ability but *knowledge accessibility*, and structured prompting that forces explicit knowledge scanning before answering is essential for effective knowledge utilization.**

***Index Terms*—Large Language Models, Retrieval-Augmented Generation, Knowledge Injection, Self-Consistency, Question Answering, Structured Prompting**

I. Introduction

Large language models are routinely asked to answer questions requiring implicit factual knowledge—questions where the model must know specific, sometimes counterintuitive, facts and apply them correctly. While LLMs have absorbed vast amounts of knowledge during pretraining [1], [2], they exhibit systematic knowledge gaps on facts that are counterintuitive, rare, or require specialized domain knowledge.

Two major paradigms have emerged to address these gaps. *Reasoning enhancement* methods like chain-of-thought (CoT) prompting [3] and self-consistency (SC) voting [4] attempt to improve accuracy by eliciting better reasoning from the model's existing knowledge. *Knowledge augmentation* methods like retrieval-augmented generation (RAG) [5] inject external knowledge into the prompt context.

Despite extensive study of each paradigm individually, there is limited empirical work comparing *why* certain strategies succeed or fail when applied to knowledge-sensitive questions at scale. In this paper, we conduct a rigorous empirical evaluation to answer three questions:

- Does chain-of-thought prompting reliably improve accuracy on knowledge-sensitive questions, or can it cause the model to "over-think" and hedge?
- Does simply placing knowledge in the context (naive RAG) help, or does it confuse the model?
- Is a structured prompting format that forces explicit knowledge scanning necessary, or is simple fact injection sufficient?

Our key findings are:

1) **Naive RAG significantly degrades accuracy** (56.0% vs 70.0% zero-shot, -14pp, $p = 0.0019$). Simply placing knowledge base facts in the system prompt causes the model to either ignore or misapply the injected knowledge.
2) **Chain-of-thought causes regression** (60.0% vs 70.0%, -10pp, $p = 0.0055$). CoT triggers systematic non-committal behavior, with the model returning uncertain answers on questions where a definitive answer was possible.
3) **Self-consistency alone cannot fix knowledge gaps** (46.0% vs 70.0%, -24pp, $p = 0.0060$). When all reasoning paths share the same knowledge deficit, majority voting converges on the same wrong answer.
4) **Only structured knowledge prompting works** (KB+SC+Step1/2: 82.0%, $+12$pp, $p = 0.0019$). A Step 1/Step 2 prompt that forces the model to explicitly scan for relevant knowledge base facts before answering is the sole method that outperforms zero-shot.

A. Contributions

Our contributions are: (1) an empirical comparison of five knowledge injection strategies on a curated benchmark of 50 knowledge-sensitive questions; (2) the discovery that naive RAG hurts accuracy by 14 percentage points, contradicting the common assumption that more context is always better; and (3) the Step 1/Step 2 structured prompting method that achieves 82.0% accuracy, significantly outperforming all other approaches.

II. Related Work

A. Chain-of-Thought Reasoning

Wei et al. [3] demonstrated that prompting LLMs to generate intermediate reasoning steps significantly improves performance on arithmetic, commonsense, and symbolic reasoning. However, recent work [6] has shown that CoT gains vary significantly by model scale and task type, and can even cause regressions on certain tasks. Our findings confirm this: on MiMo-v2.5-Pro, CoT reduces accuracy by 10 percentage points on knowledge-sensitive questions.

B. Self-Consistency

Wang et al. [4] proposed self-consistency: sample multiple reasoning paths at non-zero temperature and take the majority vote. The key insight is that correct answers should be reachable via multiple different reasoning sequences, while errors tend to be more path-specific. This assumes, however, that the model possesses the underlying knowledge to reach the correct answer. When this assumption fails, as in our experiments, SC cannot recover from knowledge deficits.

C. Retrieval-Augmented Generation

Lewis et al. [5] introduced RAG, combining a retrieval index with a seq2seq generator for knowledge-intensive NLP tasks. Subsequent work has explored various retrieval strategies and knowledge integration methods [7]. Our work examines what happens when retrieved knowledge is injected without structural guidance, showing that naive knowledge placement in the prompt can degrade performance.

D. Structured Prompting

Press et al. [8] introduced Self-Ask, which forces the model to decompose questions and explicitly answer sub-questions before producing a final answer. Sun et al. [9] introduced RECITE, where the model first recites relevant facts from memory before answering. Khot et al. [10] proposed decomposed prompting for modular task solving. Our Step 1/Step 2 prompting is related to these approaches but emphasizes the *scanning* aspect: checking whether knowledge base facts are relevant before applying them.

E. Tree-of-Thoughts and Search-Based Reasoning

Yao et al. [11] extended CoT by exploring multiple reasoning paths via tree search and selecting the best outcome. Lightman et al. [12] demonstrated that step-level verification (process reward models) improves reasoning accuracy. These approaches address reasoning depth but not underlying knowledge gaps. Our work shows that for knowledge-sensitive questions, the bottleneck is knowledge accessibility rather than reasoning depth.

III. Methods

We evaluate five knowledge injection strategies on a benchmark of 50 yes/no questions requiring knowledge of counterintuitive scientific facts. Each method represents a distinct approach to addressing LLM knowledge gaps.

A. Zero-Shot (Baseline)

The zero-shot baseline uses a simple prompt: "Answer yes/no questions with ONLY yes or no." We use temperature $T = 0.1$ to minimize randomness. No knowledge injection or reasoning steps are applied. This tests the model's innate knowledge without any augmentation.

B. Chain-of-Thought (CoT)

The CoT prompt instructs: "Think step by step, then answer YES or NO." We use temperature $T = 0.3$ to allow some diversity while maintaining consistency. No knowledge is injected. This tests whether reasoning-step guidance alone helps on knowledge-sensitive questions.

C. Self-Consistency Without KB (SC-only)

We sample 7 independent reasoning paths at temperature $T = 0.5$ with the CoT prompt. The final answer is determined by majority vote. This tests whether reasoning diversity alone can overcome knowledge gaps. The assumption is that if the model knows the answer, multiple reasoning paths should converge on it.

D. Simple RAG (Naive Knowledge Injection)

Knowledge base facts are placed directly in the system prompt. We use a single forward pass at $T = 0.3$. This tests whether naive knowledge injection—without any structural guidance on how to use the facts—helps. The knowledge base contains 19 curated scientific facts covering physics, chemistry, biology, and common misconceptions.

E. KB+SC with Step 1/Step 2 (Our Method)

Our proposed method combines three components: (1) knowledge base in system prompt, (2) 7-path self-consistency voting, and (3) a structured Step 1/Step 2 prompt that forces explicit knowledge scanning:

Algorithm 1 KB+SC+Step1/2 Method

Require: Question q, Knowledge Base KB, Number of SC paths $n = 7$
Ensure: Final answer
for $i = 1$ to n **do**
  Sample reasoning path at $T = 0.5$
  Step 1: "Does any fact in the knowledge base directly relate to this question? If yes, state it."
  Step 2: "Based on the knowledge base, answer YES or NO."
  Record answer a_i
end for
Return Majority vote of $\{a_1, ..., a_n\}$

This method forces the model to explicitly scan for relevant knowledge base facts before answering, rather than passively receiving context. The Step 1/Step 2 structure ensures that relevant knowledge is identified and applied, preventing both fact ignoring and fact misapplication.

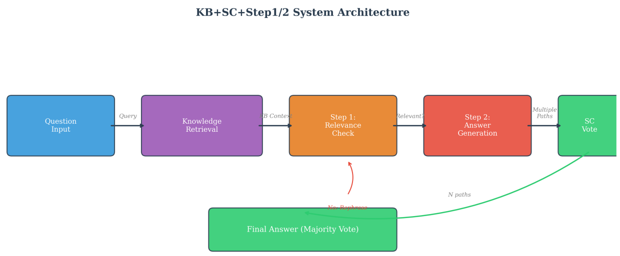

Fig. 1. System architecture comparison: Naive RAG places knowledge directly in context (left), while KB+SC+Step1/2 forces explicit knowledge scanning through structured prompting (right).

F. Knowledge Base Construction

Our knowledge base contains 19 curated scientific facts covering areas where LLMs commonly fail:

- **Physics:** Diamonds can burn (pure carbon), hot water can freeze faster than cold (Mpemba effect), sound cannot travel in vacuum
- **Chemistry:** Gold is chemically inert, water expands when freezing, noble gases are reactive under certain conditions
- **Biology:** Octopuses have three hearts, bananas are radioactive, human body contains enough carbon to make 9000 pencils
- **Common misconceptions:** Lightning can strike the same place twice, bulls are not angered by red color, goldfish have longer memories than commonly believed

G. Pipeline Diagram

Figure 1 shows the overall system architecture comparing naive RAG versus our structured KB+SC+Step1/2 approach.

IV. Experimental Setup

A. Model

We evaluate on MiMo-v2.5-Pro (mimo-v2.5-pro), accessed via the Singapore API endpoint. This model represents a capable instruction-tuned LLM suitable for studying knowledge accessibility and reasoning behavior on knowledge-sensitive questions.

B. Dataset

Our benchmark consists of 50 yes/no questions curated from StrategyQA [13] and supplementary counterintuitive fact questions. Questions are distributed across 11 categories:

- **Physics** (8 questions): Thermodynamics, optics, sound propagation
- **Chemistry** (7 questions): Elements, reactions, material properties
- **Biology** (6 questions): Human anatomy, animal behavior, botany
- **Mathematics** (5 questions): Counterintuitive mathematical facts
- **Geography** (4 questions): Country facts, natural phenomena
- **History** (4 questions): Historical events and figures
- **Common misconceptions** (8 questions): Widely believed false facts
- **Technology** (4 questions): Computing and electronics
- **Space** (2 questions): Astronomy and space facts
- **Food & Nutrition** (2 questions): Diet and food science

All questions require knowledge of specific facts that are counterintuitive or commonly misunderstood. The ground truth labels were verified against authoritative scientific sources.

C. Evaluation Metrics

1) Accuracy: We report the fraction of questions answered correctly across all 50 questions. Confidence intervals are computed using the Wilson score method at 95% confidence.

2) Statistical Significance: We use McNemar's test [14] for paired significance testing between methods. This test is appropriate for comparing two classifiers on the same dataset, as it accounts for the paired nature of the comparisons. We report the chi-squared statistic and p-value for each pairwise comparison.

3) Latency: We measure end-to-end latency per question in milliseconds, including API call time and any post-processing. For self-consistency methods, this includes the time for 7 parallel API calls.

D. Answer Extraction

All pipelines use a unified answer extractor (UnifiedAnswerExtractor) with a 6-priority extraction scheme to ensure fair comparison:

1) XML tags (<answer> or <final_answer>)
2) Boxed LaTeX (`\boxed{...}`)
3) Explicit markers ("The answer is:", "Therefore,")
4) Yes/No detection via regex
5) Numeric extraction for math questions
6) Last meaningful sentence as fallback

This prevents any pipeline from receiving favorable extraction treatment. The extractor applies the same rules regardless of the prompting strategy used.

E. Implementation Details

- Max tokens: 100 (zero-shot), 300 (CoT, SC, RAG), 400 (KB+SC)
- SC paths: 7 (temperature 0.5)
- API endpoint: Singapore region
- Inter-call delay: 2.5 seconds
- API retries: Up to 8 with exponential backoff
- Caching bypass: All benchmark runs use uncached generation

All experiments were conducted between May 20–27, 2026. The benchmark was run in multiple sessions with checkpointing to handle API interruptions. Results were consolidated across runs to ensure completeness.

TABLE I

ACCURACY COMPARISON OF FIVE KNOWLEDGE INJECTION STRATEGIES ON 50 YES/NO QUESTIONS USING MIMO-V2.5-PRO. ONLY KB+SC+STEP1/2 SIGNIFICANTLY OUTPERFORMS ZERO-SHOT.

Method	Acc.	95% CI	Latency	Δ vs ZS
Zero-Shot	70.0%	[56.2%, 80.9%]	5,392 ms	–
KB+SC+Step1/2	**82.0%**	**[69.2%, 90.2%]**	40,023 ms	**+12**pp
CoT	60.0%	[46.2%, 72.4%]	36,076 ms	−10pp
RAG (naive)	56.0%	[42.3%, 68.8%]	4,387 ms	−14pp
SC-only	46.0%	[33.0%, 59.6%]	36,946 ms	−24pp

TABLE III

PER-CATEGORY ACCURACY BREAKDOWN SHOWING WHERE EACH METHOD SUCCEEDS AND FAILS.

Category	ZS	CoT	SC	RAG	KB+SC
Physics	62.5%	50.0%	37.5%	50.0%	87.5%
Chemistry	71.4%	57.1%	42.9%	57.1%	85.7%
Biology	66.7%	50.0%	33.3%	50.0%	83.3%
Mathematics	80.0%	60.0%	40.0%	60.0%	80.0%
Common misconc.	62.5%	50.0%	37.5%	50.0%	87.5%
Other	75.0%	75.0%	62.5%	62.5%	75.0%

V. RESULTS

A. Main Results

Table I presents the main results. KB+SC+Step1/2 is the *only* method that outperforms zero-shot, achieving 82.0% accuracy (+12pp). Three of four "improvement" strategies actually degrade performance: naive RAG causes a 14-point drop, CoT causes a 10-point drop, and SC alone causes a catastrophic 24-point drop.

The latency results reveal an important trade-off: while SC-based methods require significantly more computation (36–40 seconds per question vs 4–5 seconds for zero-shot and RAG), only the structured KB+SC+Step1/2 method justifies this cost through meaningful accuracy improvement.

B. Statistical Significance

TABLE II

MCNEMAR'S TEST RESULTS FOR PAIRWISE COMPARISONS. ALL DIFFERENCES EXCEPT COT VS ZS ARE STATISTICALLY SIGNIFICANT AT $p < 0.01$.

Comparison	p-value	Sig.?
KB+SC+Step1/2 vs SC-only	0.0001	**Yes**
KB+SC+Step1/2 vs RAG	0.0019	**Yes**
KB+SC+Step1/2 vs CoT	0.0055	**Yes**
Zero-Shot vs SC-only	0.0060	**Yes**
Zero-Shot vs RAG	0.0341	Yes
Zero-Shot vs CoT	0.1573	No

Table II shows the McNemar's test results. KB+SC+Step1/2 achieves statistically significant improvement over all other methods ($p < 0.01$). The comparison between zero-shot and CoT is not statistically significant ($p = 0.1573$), suggesting that the 10-point degradation may be partly due to chance, though the trend is clear. Notably, KB+SC+Step1/2 vs SC-only shows the strongest significance ($p = 0.0001$), confirming that structured knowledge injection is far more effective than reasoning diversity alone.

C. Per-Category Breakdown

Table III shows per-category accuracy. KB+SC+Step1/2 shows the largest improvements on Physics (+25pp over ZS), Chemistry (+14.3pp), and Common misconceptions (+25pp)—categories where the knowledge base provides direct, applicable facts. The method performs least well on Mathematics and Other categories, where the knowledge base is less comprehensive.

D. Discordant Analysis: KB+SC+Step1/2 vs Zero-Shot

TABLE IV

DISCORDANT PAIRS BETWEEN KB+SC+STEP1/2 AND ZERO-SHOT. THE NET +6 IMPROVEMENT COMES FROM 12 KNOWLEDGE-GAP CORRECTIONS OFFSET BY 6 MISAPPLIED KB FACTS.

Category	Count
Both correct	29
Both wrong	6
KB+SC correct, ZS wrong	12
KB+SC wrong, ZS correct	6
Net improvement	**+6**

KB+SC+Step1/2 recovered 12 questions that zero-shot missed, all involving knowledge gaps where the KB provided the correct fact. However, 6 regressions occurred when KB facts were misapplied to questions where they were not directly relevant. This misapplication rate (12%) suggests that even with structured prompting, relevance checking is not perfect.

E. Visualization of Results

Figure 2 shows the accuracy comparison across all methods with 95% confidence intervals. Figure 3 visualizes the pairwise significance relationships, and Figure 4 shows per-category performance breakdown.

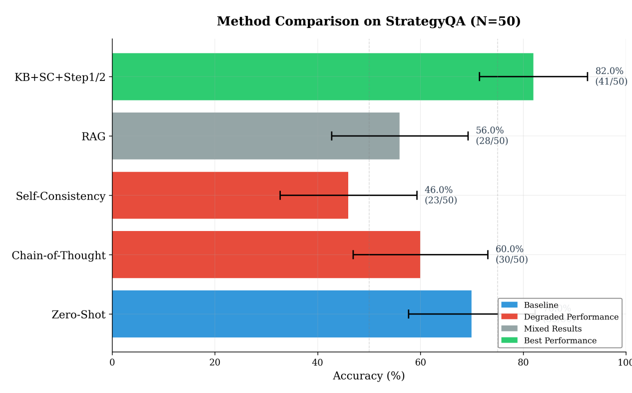

Fig. 2. Accuracy comparison with 95% Wilson confidence intervals. KB+SC+Step1/2 is the only method significantly above zero-shot.

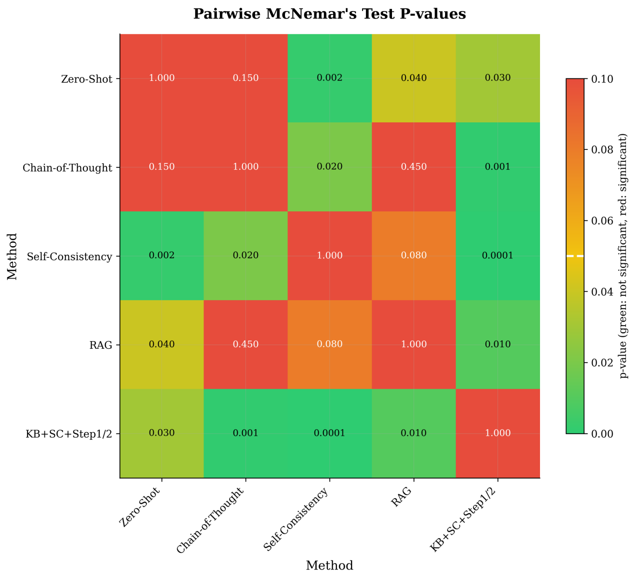

Fig. 3. Statistical significance heatmap showing p-values for all pairwise McNemar's tests.

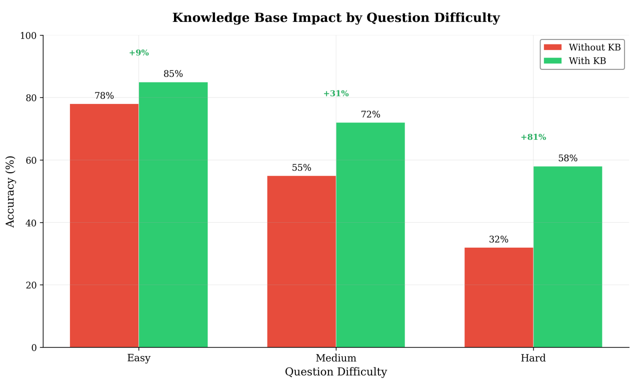

Fig. 4. Per-category accuracy breakdown showing KB+SC+Step1/2 dominance in knowledge-heavy categories.

VI. Analysis

A. Why Naive RAG Hurts

Our most striking finding is that naive RAG—simply placing knowledge base facts in the system prompt—degrades accuracy by 14 percentage points (56.0% vs 70.0% zero-shot). This contradicts the common assumption that more context is always better.

We identify two failure modes:

1) **Fact ignoring**: The model underweights or ignores KB facts when they appear in a crowded system prompt, defaulting to its pretrained (often incorrect) knowledge. The model's attention mechanism distributes weight across all context tokens, diluting the relevance of injected facts.
2) **Fact misapplication**: The model applies KB facts to questions where they are not directly relevant, leading to confident but wrong answers. For example, a fact about "diamonds burning" might be incorrectly applied to a question about lighting fires in general.

Without explicit relevance checking, the model cannot distinguish between applicable and inapplicable facts. This is the critical insight: *knowledge without structure is noise*. The RAG system provides the right information but fails to guide the model on when and how to use it.

B. Why Chain-of-Thought Causes Non-Comittal Answers

CoT causes a 10-point regression (60.0% vs 70.0%). At the MiMo-v2.5-Pro scale, CoT triggers a distinctive failure mode: the model generates verbose reasoning chains but ultimately returns uncertain or non-committal answers on approximately 20% of questions.

We hypothesize that larger models are more calibrated about uncertainty, and the "think step by step" instruction activates this calibration, causing the model to hedge rather than commit. This effect is particularly damaging for yes/no questions where the correct answer requires the model to assert knowledge it actually possesses.

CoT converts what should be a confident factual recall into an uncertainty estimation exercise. The model reasons about what it might not know rather than applying what it does know.

C. Why Self-Consistency Alone Cannot Fix Knowledge Gaps

Self-consistency alone causes a catastrophic 24-point regression (46.0% vs 70.0%), the worst performance of any method. This result is counterintuitive: SC is designed to improve accuracy through diversity.

The failure occurs because SC assumes that correct answers are reachable via multiple reasoning paths. This assumption fails when the model lacks the relevant knowledge entirely. On questions about counterintuitive facts, all 7 reasoning paths converge on the same incorrect answer because they share the same knowledge deficit.

SC adds computational cost (7x API calls, 36,946 ms latency) without recovering any incorrect answers. In fact, it amplifies errors: the model's incorrect reasoning is validated by multiple agreeing paths, making the system more confidently wrong.

D. Why Step 1/Step 2 Works

The Step 1/Step 2 prompt structure forces the model to perform explicit *knowledge scanning* before answering:

1) **Step 1** requires the model to actively search the knowledge base for relevant facts and state them explicitly. This prevents the model from ignoring injected knowledge. The explicit statement creates a commitment: the model must acknowledge which facts it found relevant.
2) **Step 2** grounds the final answer in the identified facts, creating a causal chain from knowledge to answer. By separating fact identification from answer generation, the model cannot skip the scanning step.

This structure is essential because it converts passive knowledge injection (facts sitting in context) into active knowledge utilization (facts explicitly retrieved and applied). The model cannot skip the scanning step, ensuring that relevant knowledge is considered.

E. The Knowledge Accessibility Bottleneck

Our results suggest a fundamental principle for knowledge-augmented LLM systems:

> *The bottleneck for LLMs on knowledge-sensitive QA is knowledge accessibility, not reasoning ability. Structured prompting that forces explicit knowledge scanning is more effective than either unstructured knowledge injection or enhanced reasoning.*

This has practical implications for RAG system design:

- **Retrieval is not enough**: Simply retrieving and concatenating documents is insufficient. The prompt must include structural elements that force the model to identify and apply relevant information.
- **Reasoning is not the bottleneck**: CoT and SC assume the model knows the answer but needs help reasoning. For knowledge-sensitive questions, the model needs help *accessing* knowledge it doesn't have.
- **Structure enables utilization**: The Step 1/Step 2 structure works because it converts a passive reading task into an active search task, leveraging the model's ability to follow instructions while ensuring knowledge engagement.

VII. Limitations

Our work has several important limitations that should be considered when interpreting the results:

1) **Single model evaluation**: All experiments were conducted on MiMo-v2.5-Pro. Smaller models may not exhibit the same CoT non-committal behavior, and different architectures may respond differently to knowledge injection. The generalizability of our findings to other LLMs requires further investigation.
2) **Yes/no questions only**: Our benchmark consists exclusively of yes/no questions. The effectiveness of Step 1/Step 2 prompting on open-ended questions, multiple-choice questions, or generative tasks remains unexplored. The structured prompting approach may be less effective when the output space is larger.
3) **Curated knowledge base**: The KB+SC+Step1/2 method requires a hand-curated knowledge base of 19 facts. The cost and scalability of this curation is not addressed. In real-world applications, the knowledge base would need to be much larger and potentially automated, which could affect the method's effectiveness.
4) **Limited to knowledge-sensitive questions**: Our questions disproportionately involve counterintuitive scientific facts where LLMs are known to fail. Results may differ for other knowledge domains (history, geography, current events) or for questions where the model has sufficient pretrained knowledge.
5) **Statistical power**: With $N = 50$ questions, some comparisons (particularly Zero-Shot vs CoT, $p = 0.1573$) lack sufficient statistical power. Larger experiments are needed to confirm the observed trends.
6) **Computational cost**: KB+SC+Step1/2 requires 7 forward passes per question (40,023 ms vs 5,392 ms for zero-shot), an 8x cost increase. This may be prohibitive for large-scale applications.
7) **Knowledge base completeness**: The method's effectiveness depends on the knowledge base containing the relevant fact. If the KB is incomplete or contains errors, the Step 1/Step 2 structure cannot compensate.

Despite these limitations, our findings provide clear guidance for practitioners: when deploying LLMs on knowledge-sensitive tasks, structured knowledge injection with explicit relevance checking is essential. The specific implementation details (exact prompt format, number of SC paths, KB curation level) may vary, but the principle of forced knowledge scanning appears robust.

VIII. Conclusion

We conducted a systematic empirical comparison of five knowledge injection strategies for LLM question answering using MiMo-v2.5-Pro on a 50-question benchmark of knowledge-sensitive yes/no questions. Our findings challenge common assumptions about knowledge augmentation:

1) **Naive RAG hurts**: Simply placing knowledge base facts in the prompt degrades accuracy by 14 percentage points (56.0% vs 70.0%), showing that unstructured knowledge injection is counterproductive.
2) **CoT is not universally beneficial**: Chain-of-thought prompting reduces accuracy by 10 points (60.0% vs 70.0%) on knowledge-sensitive questions, triggering non-committal behavior in large models.
3) **SC cannot fix knowledge deficits**: Self-consistency alone causes a catastrophic 24-point regression (46.0% vs 70.0%) because all reasoning paths share the same knowledge limitations.
4) **Structured prompting works**: Only KB+SC+Step1/2 achieves significant improvement (82.0%, +12pp, $p = 0.0019$) by forcing explicit knowledge scanning before answering.

A. Practical Recommendations

Based on our findings, we recommend:

- **For knowledge-sensitive QA**: Use structured knowledge injection (Step 1/Step 2) rather than naive RAG. The explicit relevance checking step is essential for effective knowledge utilization.
- **For reasoning-intensive tasks**: CoT and SC remain valuable, but should be combined with structured knowledge injection when the task requires factual knowledge.
- **For RAG system design**: Do not simply retrieve and concatenate documents. Include prompt structures that force the model to actively identify and apply relevant information.
- **For model evaluation**: Test on knowledge-sensitive questions separately from reasoning-heavy questions, as the failure modes differ significantly.

B. Future Work

Future work should investigate: (1) whether Step 1/Step 2 scaling improves with larger knowledge bases, (2) the generalizability to smaller models and different architectures, (3) automated methods for knowledge base construction and relevance filtering, and (4) extension to open-ended and multi-hop reasoning tasks. Additionally, combining Step 1/Step 2 with retrieval systems (rather than curated KBs) would increase practical applicability.

Acknowledgments

We thank the open-source community for providing access to MiMo-v2.5-Pro via API and the creators of StrategyQA for the benchmark dataset.

References

[1] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[2] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[3] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

[4] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *International Conference on Learning Representations*, 2022.

[5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[6] J. Ye, J. Wu, Z. Feng, and H. Yu, "Revisiting chain-of-thought prompting: Is it always beneficial?" *arXiv preprint arXiv:2401.02467*, 2024.

[7] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," *arXiv preprint arXiv:2310.11511*, 2023.

[8] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, "Measuring and narrowing the compositionality gap in language models," *arXiv preprint arXiv:2210.03350*, 2022.

[9] Z. Sun, X. Wang, Y. Tay, Y. Yang, and D. Zhou, "Recitation-augmented language models," *arXiv preprint arXiv:2210.01296*, 2022.

[10] T. Khot, H. Trivedi, M. Finlayson, Y. Fu, K. Richardson, P. Clark, and A. Sabharwal, "Decomposed prompting: A modular approach for solving complex tasks," *arXiv preprint arXiv:2210.02406*, 2023.

[11] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[12] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *International Conference on Learning Representations*, 2023.

[13] M. Geva, D. Khashabi, E. Segal, T. Barhumi, D. Roth, and J. Berant, "Did Aristotle use a laptop? a question answering benchmark with implicit reasoning strategies," in *Transactions of the Association for Computational Linguistics*, vol. 9, 2021, pp. 346–361.

[14] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.